



OpenSpec: Spec-Driven Development nell'era degli agenti AI

Un nuovo paradigma per la collaborazione tra umani e intelligenza artificiale

Matteo Baccan

DevFest Modena 2026 - 3/4 Ottobre 2026, Modena

Track: AI & Machine Intelligence



Google
Developer
Groups

Di cosa parleremo

01 **Dal vibe coding a OpenSpec**

Perché i prompt improvvisati non scalano e serve una struttura

03 **Il linguaggio dell'intento**

Requisiti e scenari: specifiche che l'agente non può fraintendere

05 **Nel mondo reale**

Adozione incrementale sul brownfield e anti-pattern da evitare

02 **La meccanica di OpenSpec**

Directory, artefatti e Delta Specs: l'anatomia del framework

04 **Il ciclo di esecuzione**

Propose → Apply → Archive: la macchina a stati delle modifiche

06 **Perché tutto questo conta**

Ripetibilità, intento persistente e il ruolo dell'Agentic Engineer



Google
Developer
Groups

01

Dal vibe coding a OpenSpec

Perché i prompt improvvisati non scalano e serve una struttura



Google
Developer
Groups

Il problema: agenti AI senza struttura

Gli agenti AI sono collaboratori attivi, ma senza struttura generano **vibe coding**: prompt informali, contesto volatile, drift dei requisiti.

- I requisiti vivono solo nella chat: nessuna documentazione persistente
- Allucinazioni architetturali e scelte inconsistenti
- Debito tecnico invisibile e difficilmente tracciabile



Google
Developer
Groups

La soluzione: Spec-Driven Development

Lo **Spec-Driven Development** inverte il paradigma: **la struttura prima del codice**.

	Vibe coding	Spec-Driven Development
Flusso	Prompt → Codice	Idea → Specifica → Architettura → Task → Codice
Artefatti	Nessuno (solo chat)	<code>proposal.md</code> , <code>design.md</code> , <code>tasks.md</code> , Delta Specs
Verificabilità	Impossibile	Ogni fase produce output controllabile



Google
Developer
Groups

OpenSpec: i tre pilastri

Pilastro 1: brownfield-first ($1 \rightarrow n$)

Obiettivo condiviso dai framework SDD maturi: far evolvere codebase esistenti, non solo prototipi ($0 \rightarrow 1$).

OpenSpec ci arriva con le Delta Specs: descrivi solo cosa cambia.

Pilastro 2: architettura leggera

Markdown + Git come base operativa: niente infrastruttura pesante, niente database complessi.

Il metodo resta trasparente, versionabile e vicino al flusso reale del team.

Pilastro 3: agnosticismo totale

Zero lock-in verso IDE, modelli o vendor: il valore vive nelle specifiche, non nella piattaforma.

Se cambi agente o ambiente, il processo resta intatto perché l'intento rimane nel repository.

La specifica è la fonte della verità: istruzioni vincolanti per l'agente, non suggerimenti (anche se ogni tanto i modelli se ne sbattono).

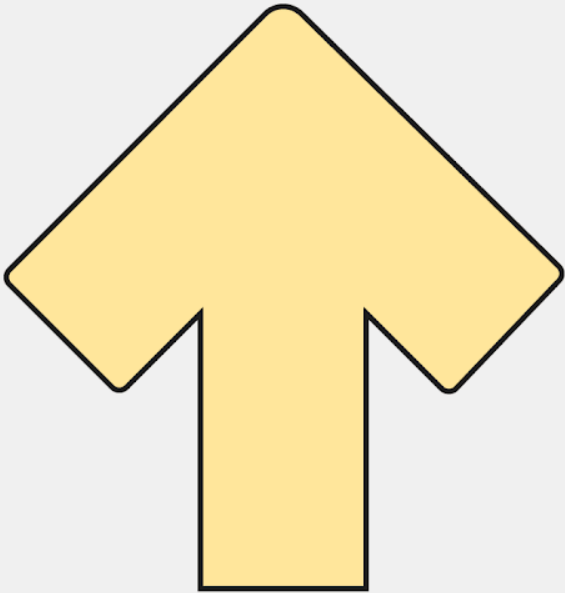


Google
Developer
Groups

02

La meccanica di OpenSpec

Directory, artefatti e Delta Specs:
l'anatomia del framework



Google
Developer
Groups

Un'unica directory: la memoria dell'agente

Tutto risiede in `openspec/`, la memoria a lungo termine dell'agente AI, con due anime separate:

Cartella	Ruolo	Contenuto
<code>specs/</code> (l'Essere)	La Source of Truth	Comportamento attuale del sistema, per domini logici (<code>auth/</code> , <code>payments/</code> , <code>ui/</code>)
<code>changes/</code> (il Divenire)	Il workspace di modifica	Una cartella isolata per ogni feature o bug fix, senza conflitti finché non si archivia

Questa distinzione garantisce transizioni sicure e modifiche parallele.



Google
Developer
Groups

Governance attiva: come `config.yaml` guida ogni richiesta

<code>config.yaml</code>	Funzione	Effetto sull'agente
<code>schema</code>	Definisce workflow e struttura degli artefatti attesi	L'agente non improvvisa formati: produce output conformi
<code>context</code>	Fornisce architettura globale, stack, standard tecnici	Ogni piano parte dagli stessi vincoli di progetto
<code>rules</code>	Impone guardrail granulari per fase e artefatto	Riduce deviazioni, allucinazioni e scelte fuori policy

Risultato operativo: il file non resta passivo nel repository, viene iniettato nel prompt di pianificazione e rende la conformità tecnica sistematica.



Google
Developer
Groups

Anatomia end-to-end di una proposta

Fase	File	Domanda chiave	Ruolo operativo
1	proposal.md	Perché / Cosa stiamo cambiando?	Definisce intento strategico e ambito lavori (business case iniziale).
2	design.md	Come lo realizziamo?	Fissa decisioni tecniche: architettura, flussi dati, librerie e vincoli.
3	tasks.md	Come eseguiamo in modo verificabile?	Scompone in task atomici e sequenziali, eseguibili dall'agente senza ambiguità.
4	Delta Specs	Cosa cambia nella Source of Truth?	Applica patch ai requisiti con sezioni ADDED , MODIFIED , REMOVED .

Flusso completo: proposal.md → design.md → tasks.md → Delta → consolidamento → specs/



Google
Developer
Groups

L'intento in pratica: `proposal.md`

Cattura l'intento strategico: "perché stiamo facendo questa modifica?". È il punto di allineamento iniziale tra umano e macchina.

Why

Gli utenti abbandonano la registrazione: serve il login social.

What Changes

- Aggiunta autenticazione OAuth (Google) al flusso di login
- Nessun impatto sul login email/password esistente



Google
Developer
Groups

Le decisioni in pratica: `design.md`

È l'ancora tecnica: fissa librerie, dati e pattern, così l'agente non "immagina" soluzioni creative ma errate.

Decisioni tecniche

- Libreria: passport con strategy passport-google-oauth20
- Nel DB solo l'id utente Google: nessun token persistito
- Sessioni: riuso del meccanismo esistente (Redis)



Google
Developer
Groups

L'esecuzione in pratica: `tasks.md`

Una checklist di azioni atomiche: l'agente aggiorna le spunte in tempo reale mentre scrive il codice, garantendo totale trasparenza.

1. Backend

- [x] 1.1 Aggiungere endpoint GET /auth/google
- [x] 1.2 Gestire il callback e creare l'utente
- [] 1.3 Test di integrazione del flusso completo



Google
Developer
Groups

Delta Specs: patch per i requisiti

La vera innovazione: invece di riscrivere l'intera specifica, l'agente descrive solo cosa cambia. Lo facciamo per ridurre il contesto da leggere e scrivere (meno token) e tenere l'agente concentrato sul cambiamento (più accuratezza).

- **ADDED Requirements**: comportamenti completamente nuovi
- **MODIFIED Requirements**: requisiti riscritti per intero, senza perdere regole pregresse
- **REMOVED Requirements**: funzionalità deprecate, con motivo e migrazione

MODIFIED Requirements

The application SHALL support 5 languages for all UI labels



Google
Developer
Groups

03

Il linguaggio dell'intento

Requisiti e scenari: specifiche che l'agente non può fraintendere



Google
Developer
Groups

Il linguaggio dell'intento: requisiti + scenari

Asse	Requisito (RFC 2119 + EARS)	Scenario (BDD)
Scopo	Ridurre ambiguità nei requisiti	Rendere verificabili i comportamenti
Forma	Parole chiave SHALL , MUST , SHOULD in frasi EARS	Struttura scenario: GIVEN , WHEN , THEN
Domanda a cui risponde	"Cosa è obbligatorio?"	"Come si osserva il risultato?"
Esito pratico	Contratti chiari per l'agente	Test di accettazione derivabili

Regola operativa: prima definiamo il vincolo con il requisito, poi ne proviamo l'esecuzione con lo scenario.



Google
Developer
Groups

Sintassi EARS

OpenSpec usa le parole chiave RFC 2119 (**SHALL**, **MUST**, **SHOULD**). Per scrivere la frase senza ambiguità: gli schemi **EARS**.

Schema	Forma
Sempre valido	The <system> SHALL <response>
Evento	WHEN <trigger> , the <system> SHALL <response>
Stato	WHILE <stato> , the <system> SHALL <response>
Comportamento indesiderato	IF <condizione> , THEN the <system> SHALL <response>

L'RFC descrive le parole chiave, EARS di Rolls-Royce struttura la frase.



Google
Developer
Groups

Dallo scenario al test

Le specifiche descrivono il comportamento osservabile, non i passi di implementazione: sono contratti di business, non tutorial di codice.

- **GIVEN:** Il contesto iniziale (es. "Dato un utente non autenticato").
- **WHEN:** L'azione scatenante (es. "Quando visita la dashboard").
- **THEN:** Il risultato atteso (es. "Allora viene reindirizzato al login").

Strutturiamo gli scenari così proprio per questo: dare all'agente una base da cui derivare i test unitari o e2e, che poi rivediamo, e chiudere il ciclo della qualità.



Google
Developer
Groups

04

Il ciclo di esecuzione

Propose → Apply → Archive: la macchina a stati delle modifiche



Google
Developer
Groups

La macchina a stati di OpenSpec

Fase	Obiettivo	Output della fase	Gate di passaggio
Propose	Allineare intento e ambito prima del codice	Cartella in <code>changes/</code> con <code>proposal.md</code> , <code>design.md</code> , <code>tasks.md</code> , Delta Specs	Revisione umana dell'intento
Apply	Implementare senza deviazioni dai vincoli	Codice + test aderenti a <code>design.md</code> e <code>tasks.md</code>	Verifica qualità su MUST/SHALL
Archive	Consolidare la modifica nella verità di sistema	Fusione Delta Specs in <code>specs/</code> + archivio storico della change	Stato aggiornato e audit trail persistente

Logica del ciclo: cattura presto il disallineamento (Propose), esegui in modo ripetibile (Apply), consolida nella Source of Truth (Archive).



Google
Developer
Groups

La revisione dell'intento

Questo è il momento chiave per lo sviluppatore umano.

- Correggere un file Markdown errato: **pochi secondi**
- Correggere un'architettura errata dopo che è stata scritta: **giorni**
- Correggerla dopo che è andata in produzione: un **avvocato**



Google
Developer
Groups

Focalizzazione: un compito alla volta

Con OpenSpec ogni fase lavora con un contesto dedicato: il modello non insegue tre obiettivi nello stesso prompt.

Fase	Focus dell'agente
Propose	Solo architettura e decisioni di design
Apply	Solo correttezza e completezza del codice
Archive	Solo coerenza documentale e Source of Truth

Più focus su un compito → Meno errori → Meno cicli di correzione



Google
Developer
Groups

Parallelismo e sincronizzazione

OpenSpec scala su due assi:

Orizzontale: branch paralleli

Più agenti su feature diverse, ognuno con la propria cartella `changes/`; il ciclo `Propose → Apply → Archive` guida il merge ordinato nella Source of Truth.

Cross-repo: Stores (beta)

La pianificazione vive in un repository dedicato e condiviso; i repo di codice si allineano dopo (openspec.dev/docs/stores).

Messaggio chiave: OpenSpec scala in orizzontale e tra repository



Google
Developer
Groups

I ruoli in un sistema multi-agente

Agente architetto

Consuma `config.yaml` e produce Record di Decisione Architetture (ADR).

Genera la struttura iniziale OpenSpec e pianifica la suddivisione logica del sistema.

Agente orchestratore

Legge il `tasks.md` prodotto dall'architetto e delega il lavoro ai team specializzati.

Supervisiona l'avanzamento senza scrivere codice direttamente.

Team di sviluppo AI

Agenti iperspecializzati per dominio: Data Access, Business Logic, Frontend.

Lavorano in parallelo sui propri task atomici, vincolati alle specifiche.

Schema d'uso, non una funzione di OpenSpec: sono gli artefatti in `changes/` a rendere possibile dividere il lavoro tra agenti.

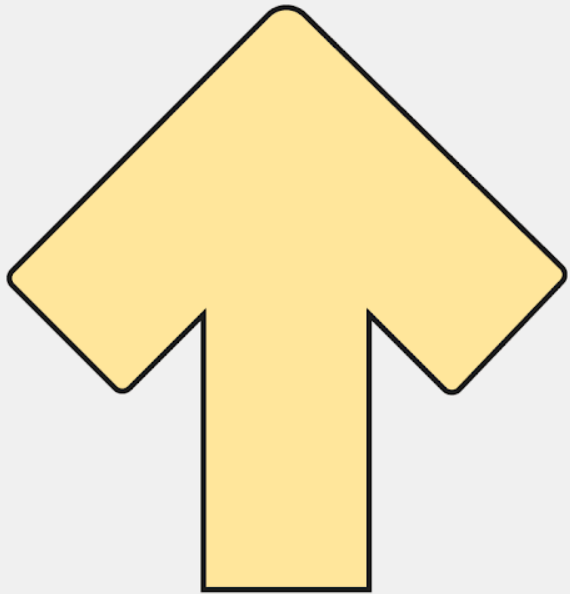


Google
Developer
Groups

05

Nel mondo reale

Adozione incrementale sul brownfield
e anti-pattern da evitare



Google
Developer
Groups

Brownfield adozione incrementale

La scala dell'adozione	Gli strumenti per partire
1. Non fermare il mondo Nessun bisogno di produrre specifiche upfront per milioni di righe.	<code>/opsx:explore</code> : l'agente legge l'area da toccare prima di proporre.
2. Sviluppo just-in-time Usa OpenSpec sulla prossima feature o sul bug critico, dove stai già intervenendo.	PRD e documenti esistenti: materiale di partenza per una Delta Spec mirata.
3. Accumulo organico La Source of Truth cresce naturalmente ad ogni ciclo <code>Propose -> Apply -> Archive</code> .	<code>/opsx:onboard</code> : tour guidato che porta una piccola modifica reale fino all'archivio.

Per partire: `npm install -g @fission-ai/openspec` e `openspec init`



Google
Developer
Groups

Mitigazione dei rischi: 4 anti-pattern da evitare

Il cimitero delle proposte

Problema: dimenticare `Archive` e lasciare change mai consolidate.

Fix: integrare l'archiviazione nel workflow operativo e nei controlli di qualità.

Il micro-management dell'AI

Problema: scrivere dettagli implementativi nelle specifiche principali.

Fix: tenere il comportamento nelle spec e spostare il "come" in `design.md`.

Burocrazia per modifiche banali

Problema: usare una SDD completa per un cambio minimo e non rischioso.

Fix: applicare progressive rigor e mantenere un approccio leggero.

Ignorare la formattazione Delta

Problema: non usare `ADDED`, `MODIFIED`, `REMOVED` in modo strutturato.

Fix: rispettare i metadati Delta per merge puliti e consolidamento affidabile.



Google
Developer
Groups

06

Perché tutto questo conta

Ripetibilità, intento persistente e
il ruolo dell'Agentic Engineer



Google
Developer
Groups

Impatto strategico

Ripetibilità + intento persistente = scalabilità umano-AI

Precisione	Resilienza	Evoluzione del ruolo
Specifiche eseguibili: più accuratezza al primo tentativo, meno rilavorazioni.	La conoscenza architettuale non muore nella chat: resta al team e resiste al turnover.	Il valore umano passa dallo "scrivere tutto" al garantire che il codice sia quello necessario.

Sintesi: quando l'implementazione è ripetibile (o, come dicono quelli bravi, *deterministica*) e l'intento rimane persistente, il sistema scala senza perdere qualità.

Il codice è l'output. La specifica è la competenza.



Google
Developer
Groups

Non solo OpenSpec: le alternative

	OpenSpec	Spec Kit (GitHub)	BMAD Method
Approccio	Change leggere su codice esistente (brownfield-first)	Una feature alla volta: <code>constitution</code> → <code>specify</code> → <code>plan</code> → <code>tasks</code> → <code>implement</code>	Metodo agile con ruoli specializzati: product, architettura, UX, sviluppo, test
Dove vive l'analisi	<code>openspec/specs/</code> + <code>openspec/changes/</code>	<code>.specify/memory/constitution.md</code> + <code>specs/NNN-feature/</code>	<code>_bmad-output/</code> : brief, PRD, architettura, epic e story
Punto di forza	Delta Specs, overhead minimo	Gate di revisione espliciti tra le fasi	Copre tutto il ciclo, dall'idea alla story

Strumenti diversi, stessa scelta di fondo: l'analisi esce dalla chat, entra nel repository e viene mantenuta insieme al codice.



Google
Developer
Groups

Cosa porti a casa da domani

- **Installa:** `npm install -g @fission-ai/openspec`, poi `openspec init` nel tuo progetto esistente
- **Parti piccolo:** documenta solo la prossima feature o il prossimo bug critico, niente Big Bang documentale
- **Scrivi per l'agente:** requisiti con `SHALL` / `MUST` (schemi EARS se servono) + scenari `GIVEN` / `WHEN` / `THEN` verificabili
- **Separa le fasi:** prima rivedi la proposta (secondi), poi lascia scrivere il codice (l'architettura sbagliata costa giorni)
- **Archivia sempre:** le Delta Specs fuse in `specs/` sono la tua Source of Truth, non la chat



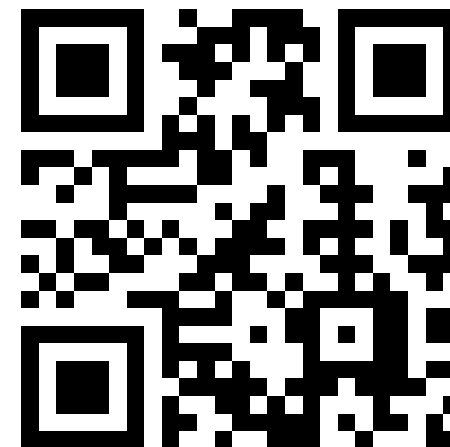
Google
Developer
Groups

Grazie! Ask Me Anything



Matteo Baccan

Rivoluzione Digitale
head of development

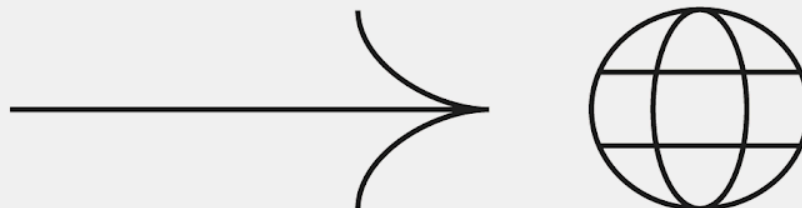


<https://www.baccan.it>

"Smetti di chattare, inizia a governare."



Google
Developer
Groups



2026



Google
Developer
Groups